

Examen

14 janvier 2020

Antoine Miné

Déroulement de l'examen

L'examen dure 2h.

Tous les documents papier sont autorisés (copies des transparents, notes de cours personnelles, livres, etc.). L'utilisation de tout moyen de communication (téléphone, tablette, etc.) est interdit. L'utilisation d'une clé USB est autorisée ; en mode examen, elle sera en lecture seule et seul le format FAT sera disponible.

Au début de l'examen, vous trouverez sur votre compte une copie des sources d'ILP dans le projet Eclipse `ILP_SU` du répertoire de travail d'Eclipse `eclipse-workspace`. Vous complèterez ce projet Eclipse en répondant aux questions de cet énoncé. Assurez-vous de placer les fichiers demandés dans le paquetage, donc dans le répertoire, précisé pour chaque question. En fin d'examen, pensez à sauvegarder tous vos fichiers, fermez Eclipse et déloguez-vous. Les fichiers de votre compte seront automatiquement ramassés.

Le point de départ de l'examen est la version **ILP 4**. Tous vos fichiers doivent être rédigés dans le paquetage `com.paracamplus.ilp4.examl920` et ses sous-paquetages du projet Eclipse `ILP_SU`. Ils se trouveront donc dans un sous-répertoire de `eclipse-workspace/ILP_SU/Java/src/com/paracamplus/ilp4/examl920` de votre compte.

L'examen sera corrigé à la main. Quand du code est demandé, les erreurs de syntaxe ne seront pas pénalisées. Il est souhaitable de n'expliciter que les parties du code qui vous semblent importantes, et de remplacer les parties peu informatives par un commentaire indiquant succinctement ce qui a été omis (c'est en particulier le cas du code répétitif, dont une version complète ne sera donnée qu'une seule fois).

L'examen est noté sur 22. Le barème pour chaque question n'est donné qu'à titre indicatif et pourra être adapté lors de la correction.

Mise en route

- Lancez Eclipse en tapant `eclipse` dans un terminal.
- Choisissez `eclipse-workspace` comme espace de travail Eclipse.
- Si Eclipse se bloque en essayant d'installer un connecteur SVN, tapez dans un terminal :

```
killall eclipse; killall java
```

puis relancez Eclipse.
- Si le projet `ILP_SU` n'apparaît pas dans le « Package Explorer », ouvrez le menu « File / Open Projects From Filesystem... », cliquez sur « Directory... », choisissez le répertoire `eclipse-workspace/ILP_SU`, puis « Finish ».
- Ouvrez le projet `ILP_SU` contenant les sources d'ILP4 et vérifiez qu'il compile bien.
- En cas de problème de compilation, il est possible que l'environnement Java JRE soit mal configuré. Allez dans « Project / Properties / Java Build Path / Libraries » puis :
 - sélectionnez « JRE System Library [JavaSE-1.8] (unbound) » et cliquez sur « Remove » ;
 - sélectionnez « Add Library... / JRE System Library / Workspace default JRE (jdk-10/0/1) » et cliquez sur « Finish ».

- Compilez la bibliothèque d'exécution en tapant dans un terminal :

```
cd ~/eclipse-workspace/ILP_SU/C/; make
```

En cas d'erreur, essayez de lancer `make` une deuxième fois.

- Le plug-in ANTLR devrait fonctionner sans réglage particulier.

Vous pouvez toutefois générer les grammaires à la main avec un clic droit sur le fichier `.g4`, suivi de « Run as... / Generate ANTLR Recognizer ».

Vérifiez que les fichiers Java générés sont dans le répertoire `eclipse-workspace/ILP_SU/target/generated-sources/antlr4/antlr4` (le répertoire `antlr4` apparaît bien deux fois dans ce chemin) et comportent une directive `package antlr4`.

Vérifiez enfin que la liste des répertoires source compilés par Eclipse (« Build Path ») comporte le répertoire `target/generated-sources/antlr4` (clic droit sur le projet, puis « Build Path / Configure Build Path... / onglet Source »; le répertoire `antlr4` n'apparaît qu'une seule fois ici). En cas de problème de compilation, vous pouvez modifier les options d'ANTLR avec un clic droit sur la grammaire, suivi de « Run As... / External Tools Configurations / Tools » et ajouter dans la zone « Arguments » l'option : « `-o HOME/eclipse-workspace/ILP_SU/target/generated-sources/antlr4` » où `HOME` est remplacé par votre répertoire racine `/home/.../`.

- Vous pouvez également utiliser ANTLR en ligne de commande à l'aide de l'archive fournie dans les sources d'ILP, en faisant :

```
java -jar ~/eclipse-workspace/ILP_SU/Java/jars/antlr-4.4-complete.jar \
-o ~/eclipse-workspace/ILP_SU/target/generated-sources/antlr4/ \
LE_NOM_DE_VOTRE_GRAMMAIRE.g4
```

- Ouvrez un navigateur et allez à l'URL : <http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/DLP>. Vous y trouverez notamment les transparents du cours.
- Ouvrez également <https://www-ppti.ufr-info-p6.jussieu.fr>. Vous y trouverez une copie locale de nombreuses documentations, dont notamment celle de Java (menu Support, option Documentation).

Introduction

Dans cet examen, nous construisons un nouveau type de boucle : une boucle `for` pour itérer sur une séquence construite par un itérateur. Nous utilisons une implantation à base d'objets, en nous inspirant des itérateurs et de la boucle *for-each* de Java : ¹

- un *itérateur* est un objet ayant une méthode `next()` qui retourne le prochain élément de la séquence, ou `false` s'il n'y a pas d'élément suivant;
- un *itérable* est un objet ayant une méthode `iterator()` qui retourne un nouvel itérateur.

Nous ajoutons alors la syntaxe *for-each* suivante utilisant le mot-clé `for` :

```
for var in expr1 do expr2
```

où `expr1` représentera un objet itérable, `expr2` est le corps de la boucle et `var` est une variable liée dans `expr2` aux valeurs de la séquence. L'effet de cette instruction `for` sera de :

1. évaluer l'expression `expr1` ; elle doit retourner un itérable ;
2. appeler la méthode `iterator` sans argument sur l'itérable et stocker dans un temporaire `ite` la valeur retournée ; ce doit être un itérateur ;
3. appeler la méthode `next` sans argument sur `ite` et stocker sa valeur de retour dans `var` ;
4. si la valeur de `var` est `false`, l'exécution du `for` se termine immédiatement et retourne `false` (comme pour une boucle `while`) ; sinon, l'exécution se poursuit à l'étape 5 ;
5. exécuter `expr2` ;
6. retourner à l'étape 3.

1. En Java, la boucle *for-each* s'écrit : `for (type var : itérable) corps`.

Notez que :

- la variable `var` est locale à l'expression `expr2` : elle n'existe qu'après l'évaluation de `expr1` et jusqu'à la fin de l'instruction `for`;
- contrairement aux itérateurs Java, nous n'utilisons pas de méthode `hasNext` : c'est la valeur de retour de `next` qui indique s'il faut terminer la boucle (`false`) ou la poursuivre (non `false`); `expr2` n'est jamais évaluée avec une valeur de `var` égale à `false`.

Voici un exemple de code ILP utilisant cette extension :

```
1 class RangeIterator {
2     var cur, max;
3     method next() {
4         if self.cur <= self.max then ( self.cur = self.cur + 1; self.cur - 1 )
5         else false
6     };
7 }
8 class Range {
9     var low, high;
10    method iterator() ( new RangeIterator(self.low, self.high) );
11 }
12 let x = new Range(10,14) in (
13     for i in x do ( print(i); print("/") );
14     for j in x do ( print(j); print("!") )
15 )
```

`RangeIterator` est une classe d'itérateurs itérant entre deux nombres, tandis que la classe `Range` est une classe d'itérables, créant des itérateurs de type `RangeIterator`.

L'exécution de ce code aura pour effet d'afficher la séquence : « 10/11/12/13/14/10!11!12!13!14! ».

1 Grammaire (2 points)

Écrivez une grammaire qui étend ILP4 avec la nouvelle construction : « `for var in expr1 do expr2` ».

Cette grammaire se trouvera dans le fichier `eclipse-workspace/ILP_SU/Java/src/com/paracamplus/ilp4/exam1920/grammar/ILPgrammar4exam1920.g4`.

2 AST (3 points)

Proposez une extension de l'arbre syntaxique d'ILP4 avec cette nouvelle construction.

Vous devez fournir toutes les classes AST dans le paquetage `com.paracamplus.ilp4.exam1920.ast`, mais aussi les interfaces IAST dans le paquetage `com.paracamplus.ilp4.exam1920.interfaces`. Vous fournirez également les classes et interfaces permettant d'étendre la fabrique d'AST et le visiteur d'AST.

3 Analyse syntaxique (2 points)

Écrivez dans le paquetage `com.paracamplus.ilp4.exam1920.parser` les nouvelles classes `ILPMLparser` et `ILPMLlistener` permettant de construire les nœuds ajoutés au langage.

4 Réécriture d'AST (5 points)

Notre nouvelle construction « `for var in expr1 do expr2` » peut être réécrite en utilisant les constructions existantes d'ILP4 : boucles `while`, déclarations locales `let in`, appels de méthodes.

1. Réécrivez à la main le code de l'exemple introductif :

```
1 let x = new Range(10,14) in (  
2     for i in x do ( print(i); print("/") );  
3     for j in x do ( print(j); print("!") )  
4 )
```

en un code ILP4 équivalent n'utilisant pas la construction `for` mais appelant directement les méthodes `iterator` et `next`.

Vous fournirez votre réponse dans le fichier texte : `eclipse-workspace/ILP_SU/Java/src/com/paracamplus/ilp4/exam1920/ast/exemple.txt`.

2. Écrivez un visiteur `com.paracamplus.ilp4.exam1920.ast.ForeachTransform` capable de transformer un AST ILP4 contenant des constructions `for` en un AST ILP4 équivalent ne contenant aucun `for`.

Nous ne demandons pas le code complet de cette classe : vous fournirez les méthodes qui vous semblent les plus importantes et vous pourrez omettre les méthodes répétitives, à condition de préciser informellement mais clairement (par exemple sous forme de commentaires dans le fichier Java demandé) la liste complète des méthodes que vous devriez implanter et leur fonctionnement général.

5 Interprétation (5 points)

Dans la suite de l'examen, nous repartons d'un AST ILP4 enrichi pouvant contenir la construction `for`. Nous n'utilisons donc pas le résultat de la question précédente.

Implantez un nouvel interprète dans une classe `com.paracamplus.ilp4.exam1920.interpreter.Interpreter` pour les programmes ILP4 avec boucle `for`.

6 Compilation (5 points)

Nous nous intéressons maintenant à la compilation des programmes ILP4 avec boucle `for`.

1. Donnez la liste de toutes les classes et interfaces que vous auriez à ajouter pour implanter le compilateur. Précisez leur nom et le packaging complet. Pour chaque interface, indiquez les interfaces dont elle hérite. Pour chaque classe, indiquez la classe dont elle hérite et les interfaces qu'elle implante. On ne demande pas le code Java de ces classes et interfaces.

Vous fournirez votre réponse dans un fichier texte : `eclipse-workspace/ILP_SU/Java/src/com/paracamplus/ilp4/exam1920/compiler/classes.txt`.

2. Donnez le code de la méthode du visiteur de normalisation, `Normalizer`, qui traite le nœud de la boucle `for` (les autres méthodes ne sont pas demandées).

Vous fournirez votre réponse dans le fichier : `eclipse-workspace/ILP_SU/Java/src/com/paracamplus/ilp4/exam1920/compiler/normalizer/Normalizer.java`.

3. Donnez le code de la méthode du visiteur d'analyse des variables libres, `FreeVariableCollector`, qui traite le nœud de la boucle `for` (les autres méthodes ne sont pas demandées).

Vous fournirez votre réponse dans le fichier : `eclipse-workspace/ILP_SU/Java/src/com/paracamplus/ilp4/exam1920/compiler/FreeVariableCollector.java`.

4. Donnez un schéma de compilation détaillé pour la construction `for`.

Vous fournirez votre réponse dans un fichier texte : `eclipse-workspace/ILP_SU/Java/src/com/paracamplus/ilp4/exam1920/compiler/schema.txt`. Le code Java de la classe `Compiler` n'est pas demandé.

5. Listez les fonctions de la bibliothèque d'exécution C que vous utiliseriez pour implanter la boucle `for`. Décrivez pour chacune son utilité et son fonctionnement. Précisez si vous devez enrichir la bibliothèque d'exécution et comment.

Vous fournirez votre réponse dans un fichier texte : `eclipse-workspace/ILP_SU/Java/src/com/paracamplus/ilp4/exam1920/compiler/lib.txt`.